

# Introduction to Unit Tests

My Reaction



When someone says testing is easy



# Agenda

---

Motivation

Case study

Nunit

Basic parts of a unit tests

What does a unit test look like?

Exercises (Halloween style)

Testing and pitfalls

# Manually test

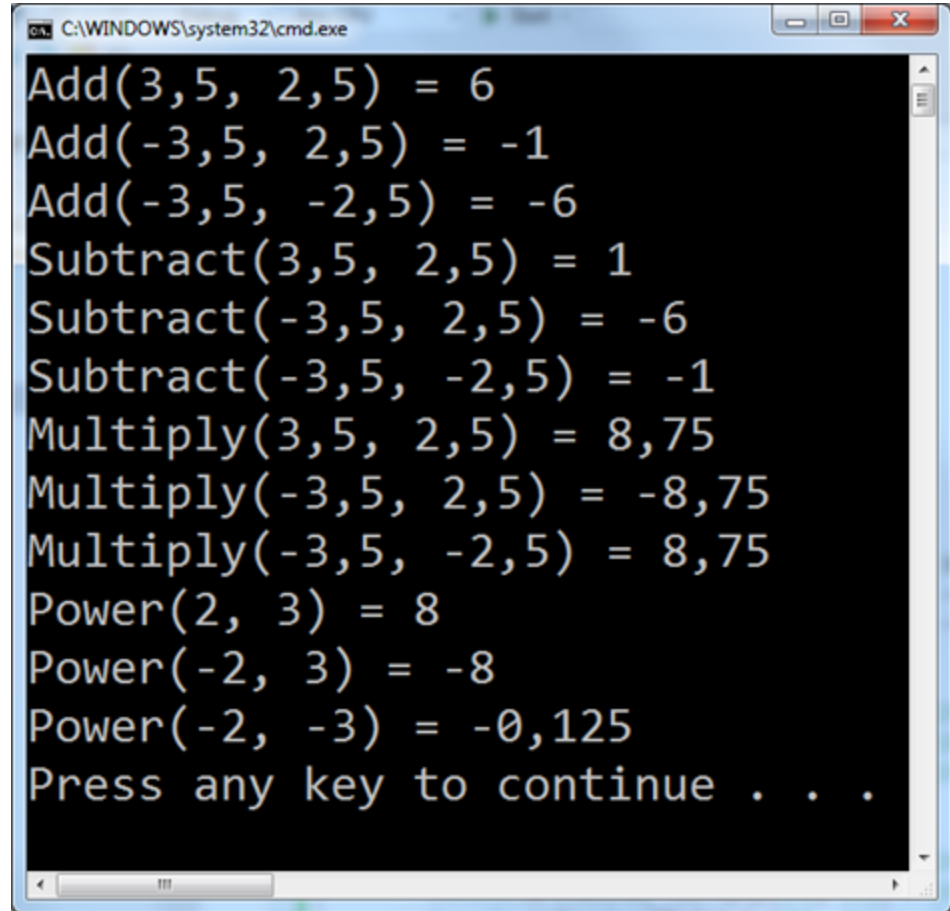
```
public class Calculator {  
    public double Add(double a, double b) {  
        return a + b;  
    }  
  
    public double Subtract(double a, double b) {  
        return a - b;  
    }  
  
    public double Multiply(double a, double b) {  
        return a * b;  
    }  
  
    public double Power(double a, double b) {  
        return Math.Pow(a,b);  
    }  
}
```

```
class Program  
{  
    static void Main(string[] args)  
    {  
        // Declare the unit-under-test  
        var uut = new Calculator();  
  
        // Test Add()  
        Console.WriteLine("Add({0}, {1}) = {2}", 3.5, 2.5, uut.Add(3.5, 2.5));  
        Console.WriteLine("Add({0}, {1}) = {2}", -3.5, 2.5, uut.Add(-3.5, 2.5));  
        Console.WriteLine("Add({0}, {1}) = {2}", -3.5, -2.5, uut.Add(-3.5, -2.5));  
  
        // Test Subtract()  
        Console.WriteLine("Subtract({0}, {1}) = {2}", 3.5, 2.5, uut.Subtract(3.5, 2.5));  
        Console.WriteLine("Subtract({0}, {1}) = {2}", -3.5, 2.5, uut.Subtract(-3.5, 2.5));  
        Console.WriteLine("Subtract({0}, {1}) = {2}", -3.5, -2.5, uut.Subtract(-3.5, -2.5));  
  
        // Test Multiply()  
        Console.WriteLine("Multiply({0}, {1}) = {2}", 3.5, 2.5, uut.Multiply(3.5, 2.5));  
        Console.WriteLine("Multiply({0}, {1}) = {2}", -3.5, 2.5, uut.Multiply(-3.5, 2.5));  
        Console.WriteLine("Multiply({0}, {1}) = {2}", -3.5, -2.5, uut.Multiply(-3.5, -2.5));  
  
        // Test Power()  
        Console.WriteLine("Power({0}, {1}) = {2}", 2.0, 3.0, uut.Power(2.0, 3.0));  
        Console.WriteLine("Power({0}, {1}) = {2}", -2.0, 3.0, uut.Power(-2.0, 3.0));  
        Console.WriteLine("Power({0}, {1}) = {2}", -2.0, -3.0, uut.Power(-2.0, -3.0));  
    }  
}
```



# Validate result?

---



A screenshot of a Windows command prompt window titled "C:\WINDOWS\system32\cmd.exe". The window has a black background with white text. It displays a series of mathematical calculations, each on a new line. The calculations are: Add(3,5, 2,5) = 6, Add(-3,5, 2,5) = -1, Add(-3,5, -2,5) = -6, Subtract(3,5, 2,5) = 1, Subtract(-3,5, 2,5) = -6, Subtract(-3,5, -2,5) = -1, Multiply(3,5, 2,5) = 8,75, Multiply(-3,5, 2,5) = -8,75, Multiply(-3,5, -2,5) = 8,75, Power(2, 3) = 8, Power(-2, 3) = -8, and Power(-2, -3) = -0,125. The last line of the output is "Press any key to continue . . .". The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

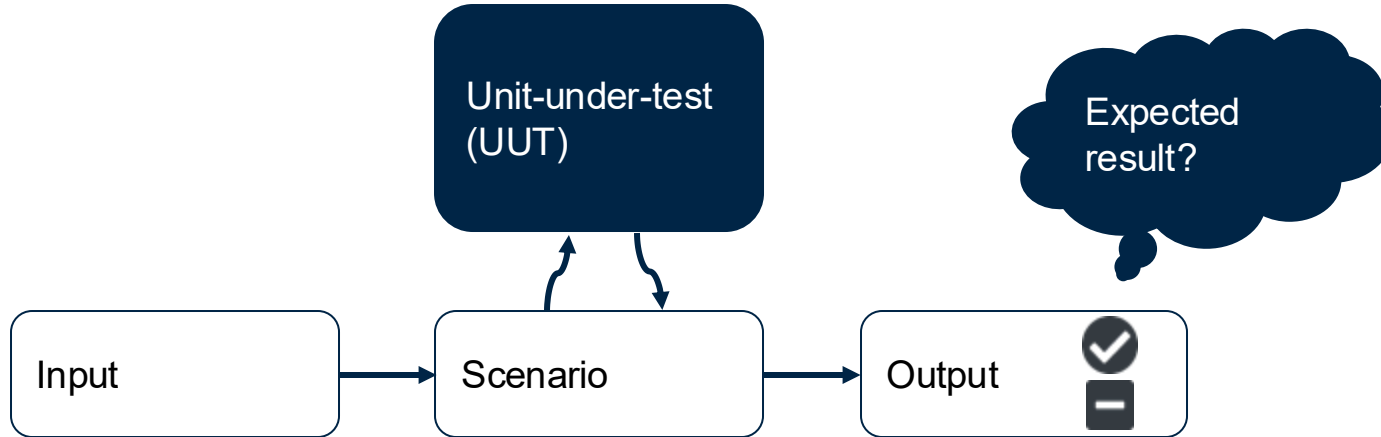
```
C:\WINDOWS\system32\cmd.exe
Add(3,5, 2,5) = 6
Add(-3,5, 2,5) = -1
Add(-3,5, -2,5) = -6
Subtract(3,5, 2,5) = 1
Subtract(-3,5, 2,5) = -6
Subtract(-3,5, -2,5) = -1
Multiply(3,5, 2,5) = 8,75
Multiply(-3,5, 2,5) = -8,75
Multiply(-3,5, -2,5) = 8,75
Power(2, 3) = 8
Power(-2, 3) = -8
Power(-2, -3) = -0,125
Press any key to continue . . .
```





# A unit tests

---



Icons: FrICONiX.com

# Scenario

---

Can be a requirement for the method/class

Different types:

Positive scenario (happy path)

- Expected behavior

Negative scenario

- Invalid / unexpected input

Edge case scenario

- Boundary input

Exception scenario

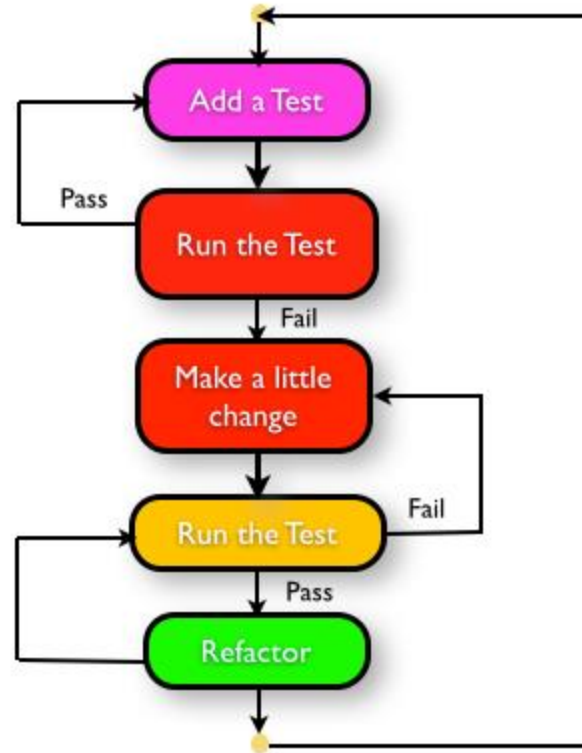
- Exception are thrown as expected

# How to plan and execute test

1. Choose a scenario
2. Define input and output
3. Write the [TestCase]
4. Run the test\*
5. Implement the business logic
6. Run the test\*\*
7. Goto 1

\* to test it fails

\*\* to validate test succeeds





# Demo: A cash register

---

We will implement and test a class CashRegister

| CashRegister             |
|--------------------------|
| + AddItem(price: double) |
| + GetNItems(): int       |
| + GetTotal(): double     |



**Your** turn to plan the tests: What test cases do we need for each of the class' methods?

What is the scenario?

What is the test input?

What is the expected result?

# Test also gives...

---

specifications built into the program

confidence in code

early error finding

decoupled system

better design



# Test framework

---

Gives:

Support for automation

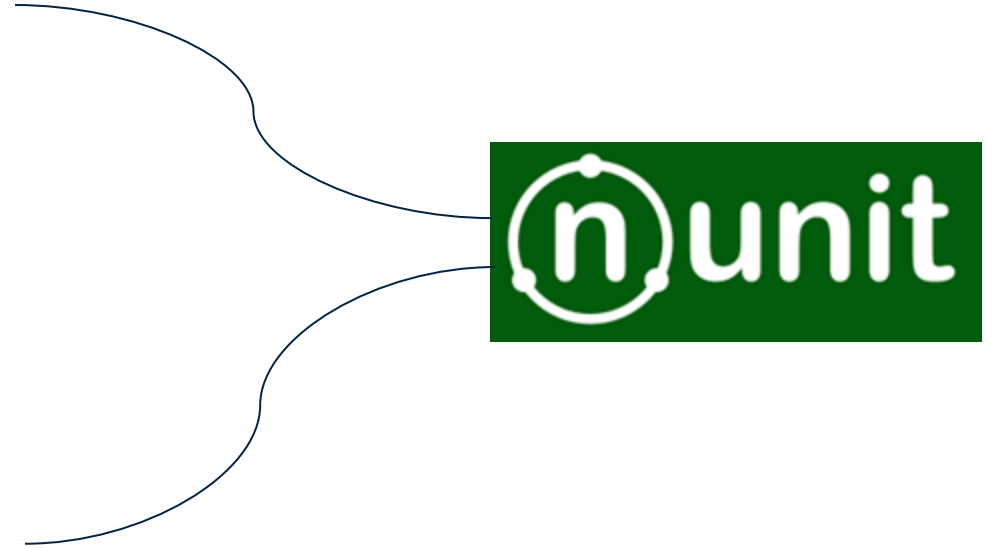
Easy setup and removal

Good assertion constructs

Detailed test reports

Nice IDE integration

Testing styles



# Automation / test report

The screenshot displays the Visual Studio Test Explorer on the left and the Code View on the right. The Test Explorer shows a list of tests under 'MultiTest (26)'. The tests are categorized by status: failed (red X), warnings (yellow triangle), and passed (green checkmark). The 'TestMethodN12' is highlighted with a red arrow. The Code View shows the source code for 'MSUnitTestBase' with a [TestClass] attribute and a [TestMethod] attribute. The output window at the bottom shows the test results, including the start and finish times of the run.

| Test Name           | Duration | Status  |
|---------------------|----------|---------|
| TestMethodMSOutput1 | 19 ms    | Failed  |
| TestMethodMs4       |          | Warning |
| TestMethodMs5       |          | Warning |
| AsyncTestMS1        | 1 ms     | Passed  |
| BaseTestMethod1     | < 1 ms   | Passed  |
| DerivedTestMethod   | < 1 ms   | Passed  |
| NameTest            | 3 ms     | Passed  |
| TestAddInNet4       | 2 ms     | Passed  |
| TestMethodMs1       | 2 sec    | Passed  |
| TestMethodMs11      | 201 ms   | Passed  |
| TestMethodMs12      | < 1 ms   | Passed  |
| TestMethodMs2       | 2 sec    | Passed  |
| TestMethodMs3       | 3 sec    | Passed  |
| TestMethodMs6       | < 1 ms   | Passed  |
| TestMethodMSOutput2 | < 1 ms   | Passed  |
| TestMethodN1        |          | Passed  |
| TestMethodN11       |          | Passed  |
| TestMethodN12       |          | Passed  |
| TestMethodN2        |          | Passed  |
| TestMethodN3        |          | Passed  |
| TestMethodX1        |          | Passed  |

```
[TestClass]
1 reference
abstract public class MSUnitTestBase
{
    [TestMethod]
    0 references
    public void BaseTestMethod1()
    {
        Assert.IsTrue(true);
    }
}
```

Output

Show output from: Tests

----- Run test started -----

===== Run test finished: 18 run (0:00:07,9559915) =====

# Nunit example

---

```
[TestFixture]
public class UnitTest1 {
    Calculator uut;
    [Setup]
    public void Setup() {
        uut = new Calculator();
    }

    [Test]
    public void Test_AddMethod() {
        double res = uut.Add(3.5, 2.5);
        Assert.AreEqual(res, 6);
    }
    ...
}
```



# NUnit assertion - constraints

---

Constraint-based assert model

```
Assert.That(uut.Count, Is.EqualTo(10));
```

Actual state

Constraint with  
expected state

More constraints <https://nunit.org/docs/2.5/constraintModel.html>

# NUnit assertion - constraints

Constraint-based assert model

```
Assert.That(uut.Count, Is.EqualTo(10));
```

Actual state

Constraint with  
expected state

Other constraints examples - many more exists

```
Assert.That(uut.Count, Is.GreaterThan(3));  
Assert.That(myString, Is.EqualTo("Hello"));  
Assert.That(array, Has.Exactly(3).LessThan(100));
```

More constraints <https://nunit.org/docs/2.5/constraintModel.html>



# Test framework

---

**Assertions** Different ways to compare expected and actual test results in a readable way

**Test case** Each test case - tests one specific, isolated aspect of the unit-under-test

**Test fixture** Collects test cases and helps with setup, teardown, etc.

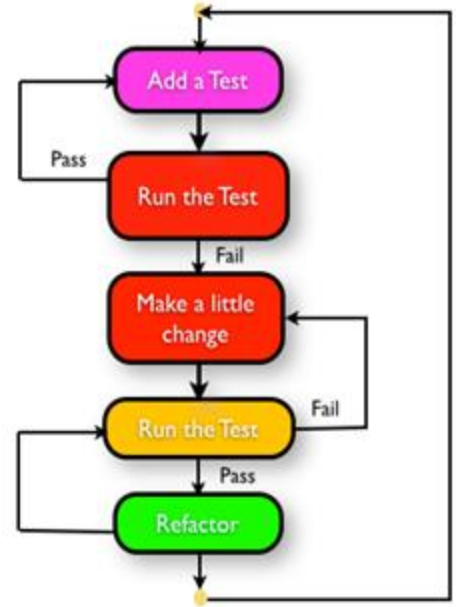
**Test runner** Runs the tests and reports the result

**Test reports** Results of the tests run

# Demo: A cash register

Let us implement the CashRegister and the TestFixture

| CashRegister             |
|--------------------------|
| + AddItem(price: double) |
| + GetNItems(): int       |
| + GetTotal(): double     |



# Your turn

---

## Exercises

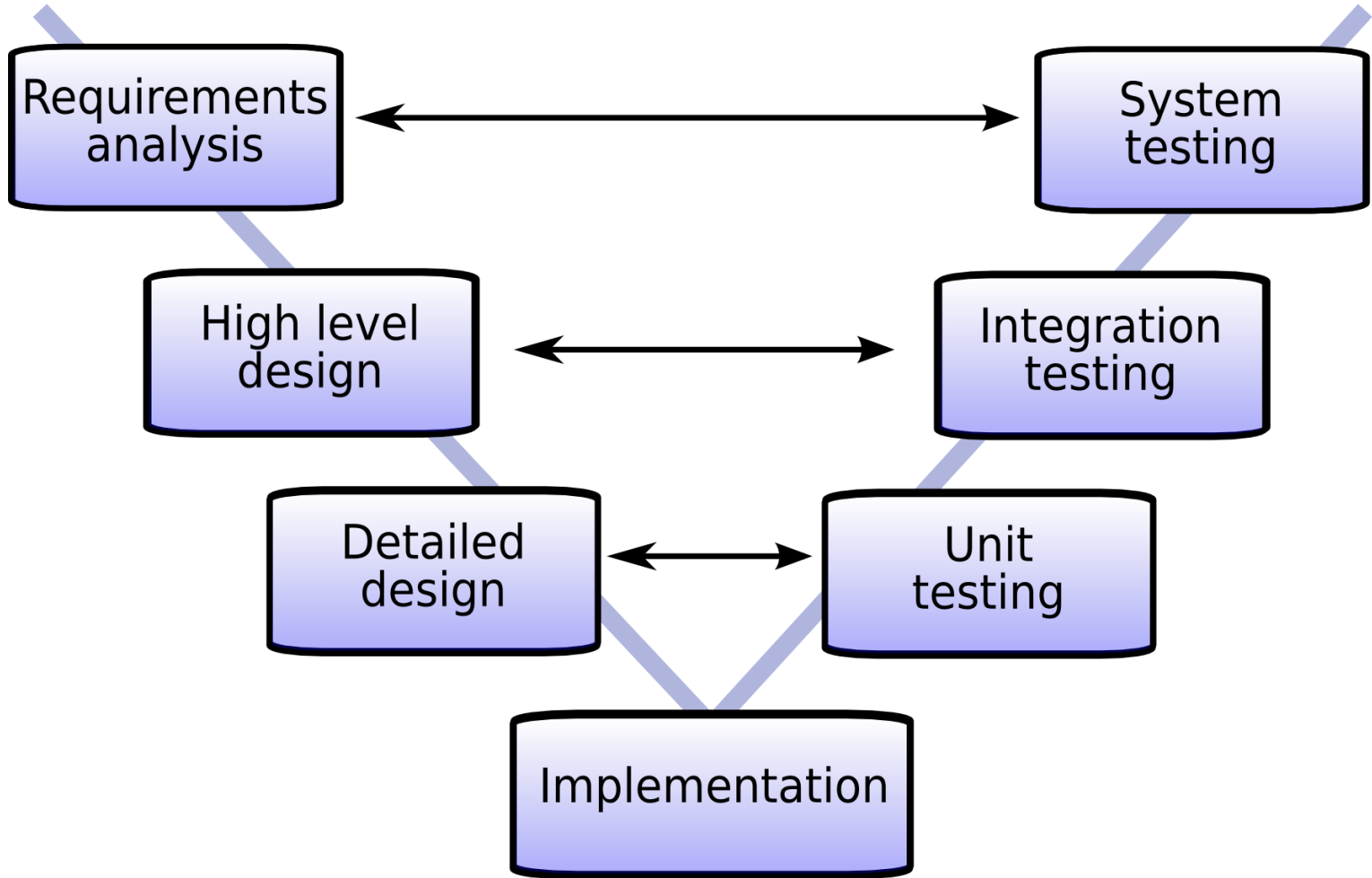
*Note: We summarize at the end of the lecture*



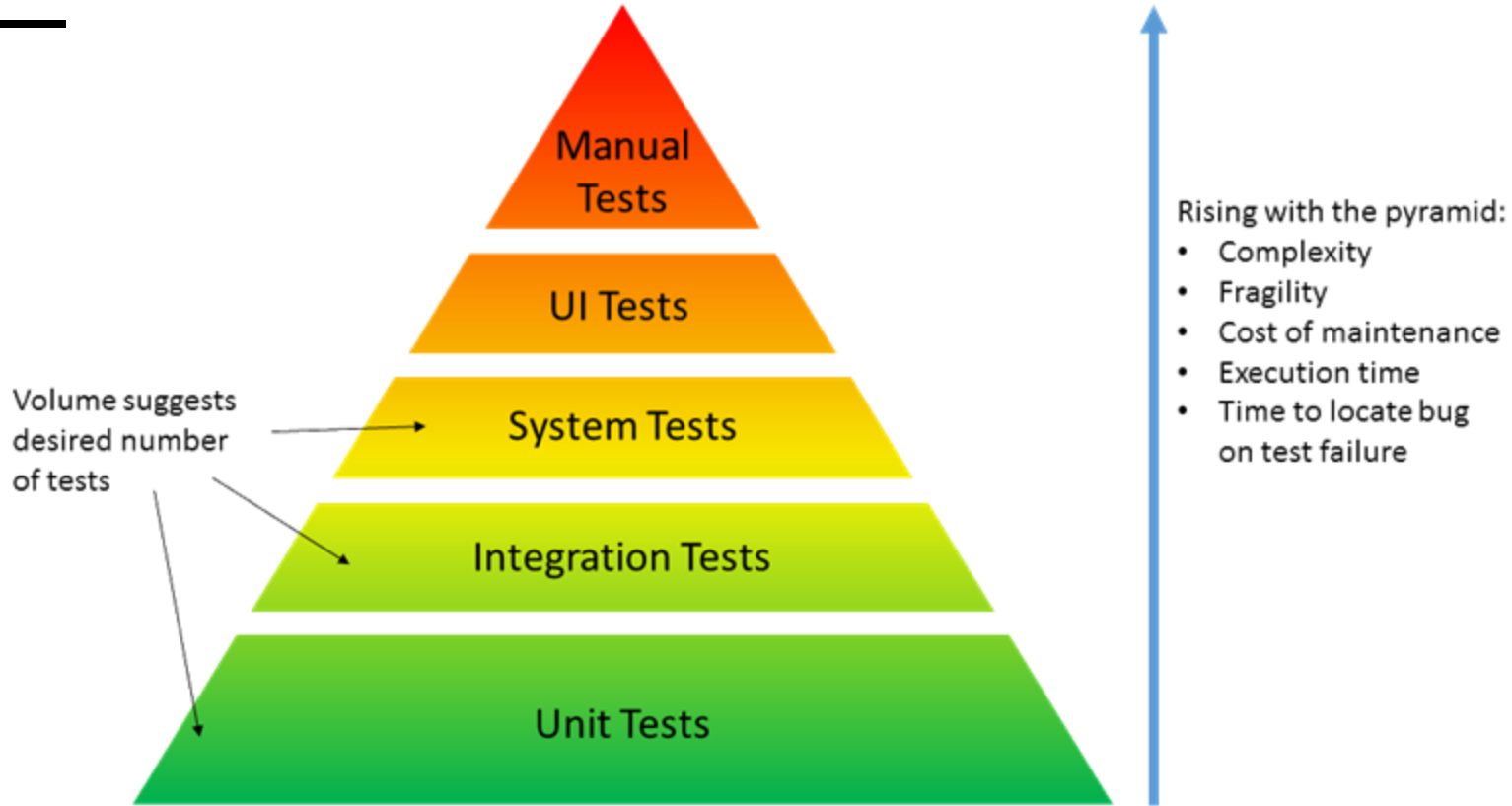
---

# Testing

## and Pitfalls



# Why test? Why unit test?



# Pitfalls

---

The true unit test contains information about the design and behavior of UUT (Unit-under-test)

**Do not** make any assumptions about other parts

# Pitfalls

---

The true unit test contains information about the design and behavior of UUT (Unit-under-test)

**Do not** make any assumptions about other parts

Integration-test **do not** tell anything about how the code base is broken down into units

Make assumptions about the whole system's behavior





# Pitfalls

The true unit test contains information about the design and behavior of UUT (Unit-under-test)

**Do not** make any assumptions about other parts

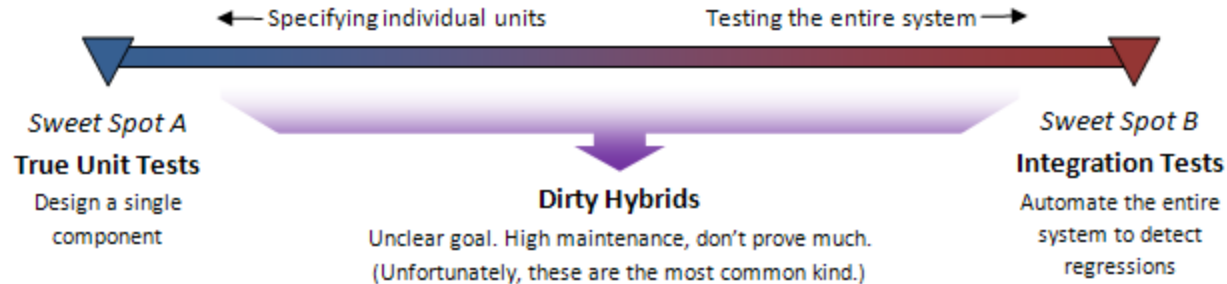
Integration-test **do not** tell anything about how the code base is broken down into units

Make assumptions about the whole system's behavior

In between

Small changes break unrelated test

Tests break - but the system works as “expected”





# Reference

---

TDD: [agilefaqs.com/services/training/test-driven-development](https://agilefaqs.com/services/training/test-driven-development)

Dancing man: <https://giphy.com>